

20 Exercises — Pointers, Functions & Procedures (Algo + C)

Statements with solutions in pseudo-algorithm and C language

Exercise 1 — Basic Pointer and Dereferencing

Statement. Read an integer x , create a pointer p to x , display x , the address of x , then increment x through the pointer and display again.

Exercise 2 — Swap Procedure Using Pointers

Statement. Write a procedure `echanger(a,b)` that swaps two integers.

Exercise 3 — Min & Max Using Output Parameters

Statement. Function that finds the min and max of an array and returns them via output parameters.

Exercise 4 — Sum and Average by Pointer Iteration

Statement. Write `somme(T,n)` and `moyenne(T,n)` by advancing a pointer.

Exercise 5 — In-Place Array Reversal (Two Pointers)

Statement. Reverse an array of integers using two pointers `left` and `right`.

Exercise 6 — String Length (Homemade strlen)

Statement. Write `len(s)` that counts up to the `'\0'` character.

Exercise 7 — String Copy (Homemade strcpy)

Statement. Write `copier(dst,src)` that copies until `'\0'`.

Exercise 8 — Counting Vowels with Pointers

Statement. Return the number of vowels in a string (case-insensitive).

Exercise 9 — Remove Spaces (Two Pointers : Read/Write)

Statement. Remove all spaces from a string in place.

Exercise 10 — Pointer to the Minimum Element

Statement. Return the *address* of the smallest element ; then compute its index.

Exercise 11 — Dynamic Allocation and Average

Statement. Dynamically allocate n real values, read them, compute the average, free the memory.

Exercise 12 — Flattened Matrix (Pointer Indexing)

Statement. Store an $N \times N$ matrix in a 1D array and write `get(A,N,i,j)` and `set(A,N,i,j,val)`.

Exercise 13 — Append to Dynamic Array (Double Pointer)

Statement. Write `push_back(int **T,int *n,int v)` which performs `realloc` and adds v .

Exercise 14 — Apply a Function to Each Element (Callback)

Statement. Write `appliquer(T,n,f)` where f is `int f(int)`.

Exercise 15 — Sorting with qsort and Comparison Function Pointer

Statement. Sort an array of integers in increasing and then decreasing order using two comparators.

Exercise 16 — Structure and Pointer to Structure

Statement. Define `struct Etudiant{nom[32]; note;}` and a procedure `init(E,nom,note)`.

Exercise 17 — Linked List : Insert at Head

Statement. Implement `push_front(&head, val)`.

Exercise 18 — Split a Line by Separator (Pointer-Based Tokens)

Statement. Transform `s` into tokens separated by `,` by placing `'\0'` and storing the addresses in `out[]`. Return the number of tokens.

Exercise 19 — Dot Product of Two Vectors (Pointers & const)

Statement. Write `dot(const int*A,const int*B,int n)` by iterating with pointers.

Exercise 20 — Trapezoidal Integration with Function Pointer

Statement. Write `integrale(f,a,b,n)` (trapezoidal rule) where `f` is `double f(double)`.